

**SKILLING-ML/DL****II-II CSE, CSE(AI&ML)****AY: 2025-2026**

SL No	Week	Slot-1 (3 Hours)	Detailed Subtopics	Slot-2 (3 Hours)	Detailed Subtopics
1	Week 3	Advanced Supervised Learning & Metrics Tree-based & Ensemble Models(Theory)	Decision Trees, Random Forests, SVM, Ensemble learning Decision tree construction, Splitting criteria (Gini, Entropy), Random Forest algorithm, Ensemble learning principles, Overfitting control in trees	Model Evaluation Metrics Performance Evaluation Metrics	Accuracy, Precision, Recall, F1-score, ROC-AUC, Confusion matrix model development on dataset, Confusion matrix, Accuracy, Precision, Recall, F1-score, ROC curve, AUC score, Model comparison using metrics

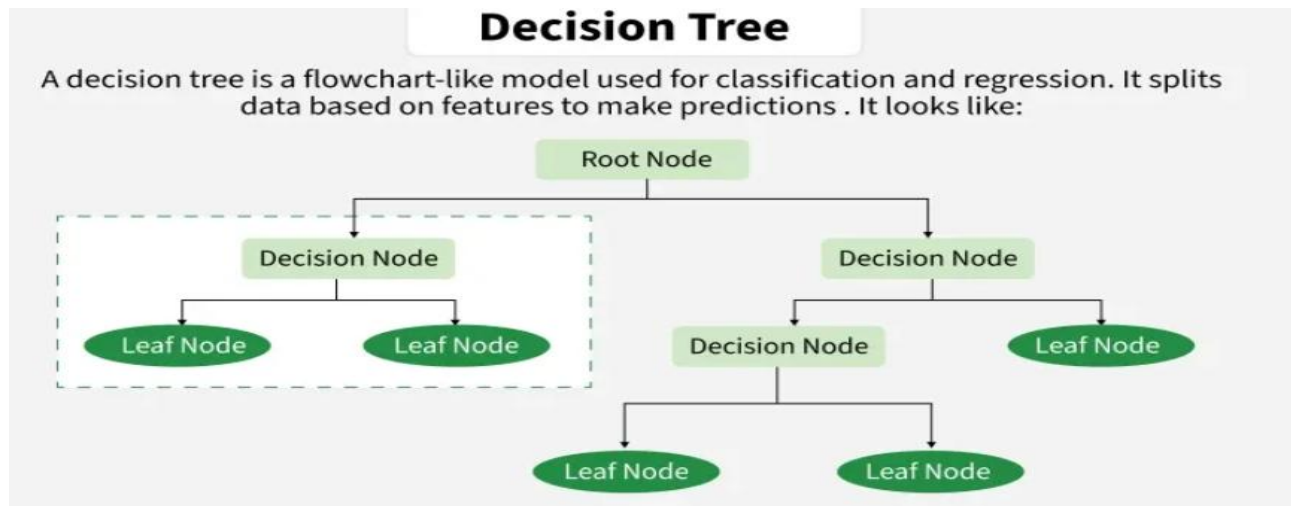
WEEK 3 – Session 1: Advanced Supervised Learning & Metrics Tree-based & Ensemble Models (Theory)

1. Decision trees
2. Splitting criteria (Gini, Entropy)
3. Random Forest algorithm
4. SVM
5. Naive Bayes
6. Overfitting control in trees

1) Decision Trees

A decision tree is a supervised learning algorithm used for both classification and regression tasks. It has a hierarchical tree structure which consists of a root node, branches, internal nodes and leaf nodes. It works like a flowchart help to make decisions step by step where:

- Internal nodes represent attribute tests
- Branches represent attribute values
- Leaf nodes represent final decisions or predictions.



1.1 Structure of a Decision Tree

1. Root Node:

1. The starting point of the tree.
2. Represents the entire dataset and splits it based on the best decision rule.

2. Decision Nodes:

1. Intermediate nodes where features are evaluated, and the dataset is split into subsets.
2. Each split corresponds to a condition on a feature.

3. Leaf Nodes:

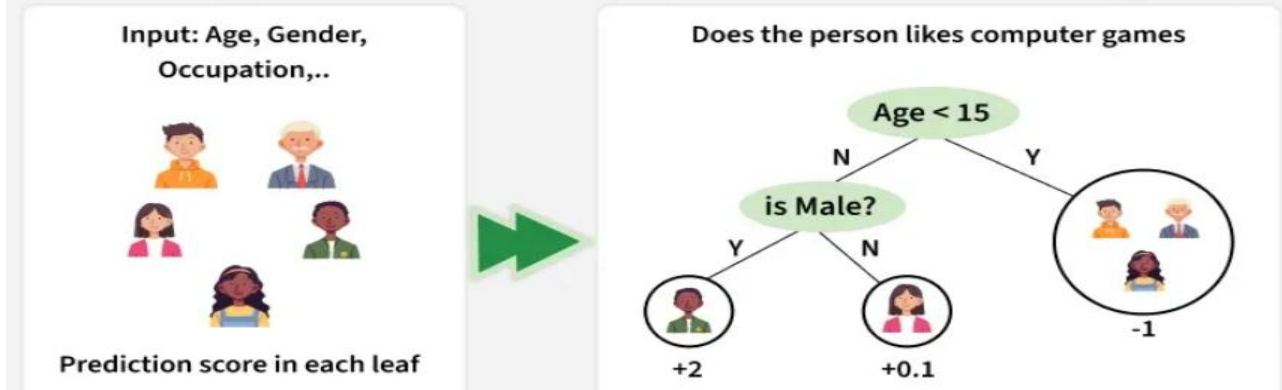
1. Terminal nodes that provide the output (class label or value).

4. Branches:

1. Pathways connecting nodes, representing the decisions or outcomes of a condition.

Working of Decision Tree

The model checks conditions like age and gender to split users into groups. Each group (leaf node) gets a prediction score based on user preferences for computer games.



1.2 Types of Decision Tree

- **ID3:** This algorithm measures how mixed up the data is at a node using something called entropy. It then chooses the feature that helps to clarify the data the most.
- **C4.5:** This is an improved version of ID3 that can handle missing data and continuous attributes.
- **CART:** This algorithm uses a different measure called Gini impurity to decide how to split the data. It can be used for both classification (sorting data into categories) and regression (predicting continuous values) tasks.

ID3 (Iterative Dichotomiser 3)

- Used only for classification
- Uses Entropy and Information Gain to choose splits
- Allows multi-way splits
- Works best with categorical data
- Does not handle pruning or continuous attributes well

CART (Classification and Regression Tree)

- Used for both classification and regression
- Uses Gini Index (classification) and MSE (regression)
- Allows only binary splits
- Handles continuous and categorical data
- Supports pruning, reducing overfitting

1.3 How Does a Decision Tree Work

A decision tree splits the dataset based on feature values to create pure subsets ideally all items in a group belong to the same class. Each leaf node of the tree corresponds to a class label and the internal nodes are feature-based decision points.

1. **Select the Best Feature to Split:**

1. Use a splitting criterion like **Gini Index, Information Gain**.
2. Choose the feature that best separates the data into pure subsets.

2. **Split the Dataset:**

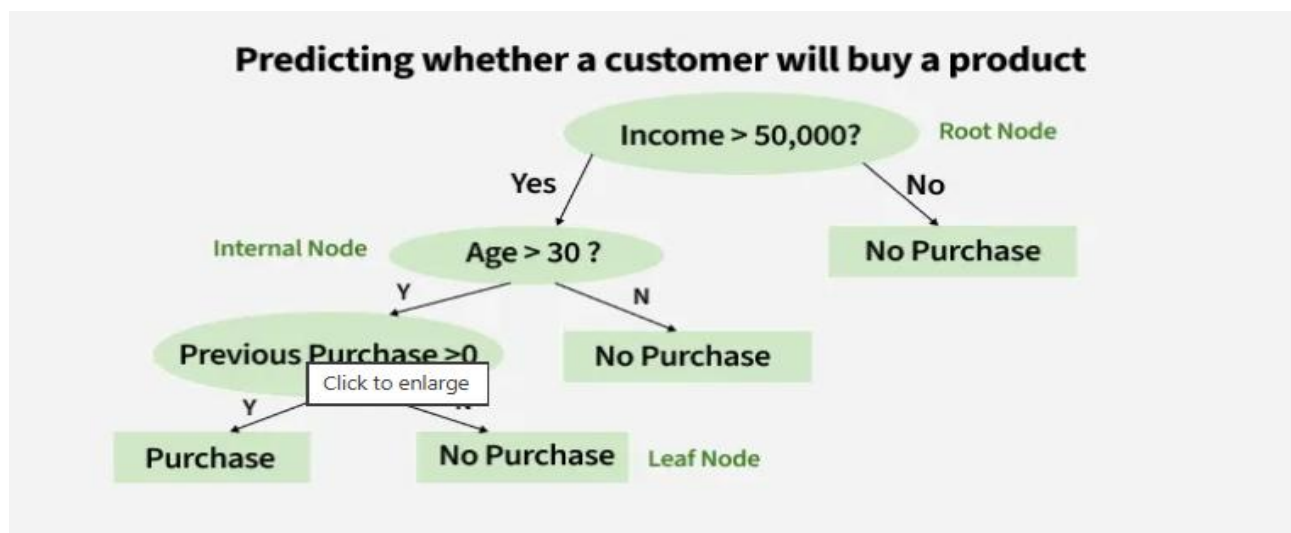
1. Divide the dataset based on the selected feature and its values.

3. **Repeat Recursively:**

1. For each subset, repeat the process until:
 1. The tree reaches a maximum depth.
 2. The dataset in a node is "pure" (only one class remains).
 3. A minimum number of samples per leaf node is reached.

4. **Output:**

1. For classification: Assign the majority class in the leaf node.
2. For regression: Assign the average value in the leaf node.



Splitting Criteria In Decision Tree

In decision trees, splitting criteria help decide which feature to split on at each node. The two most common criteria are:

Gini Index

$$I_G = 1 - \sum_{j=1}^c p_j^2$$

p_j : proportion of the samples that belongs to class c for a particular node

Entropy

$$I_H = - \sum_{j=1}^c p_j \log_2(p_j)$$

p_j : proportion of the samples that belongs to class c for a particular node.

*This is the the definition of entropy for all non-empty classes ($p \neq 0$) The entropy is 0 if all samples at a node belong to the same class.

1.4 Problem Statement

We want to decide **whether to Play Tennis or Not** based on weather conditions.

Dataset

Day	Outlook	Temperature	Humidity	Wind	Play Tennis
1	Sunny	Hot	High	Weak	No
2	Sunny	Hot	High	Strong	No
3	Overcast	Hot	High	Weak	Yes
4	Rain	Mild	High	Weak	Yes
5	Rain	Cool	Normal	Weak	Yes
6	Rain	Cool	Normal	Strong	No
7	Overcast	Cool	Normal	Strong	Yes
8	Sunny	Mild	High	Weak	No
9	Sunny	Cool	Normal	Weak	Yes
10	Rain	Mild	Normal	Weak	Yes
11	Sunny	Mild	Normal	Strong	Yes
12	Overcast	Mild	High	Strong	Yes
13	Overcast	Hot	Normal	Weak	Yes
14	Rain	Mild	High	Strong	No

Step 1: Entropy of the Target (Play Tennis)

Count Yes / No

- Yes = 9
- No = 5
- Total = 14

Entropy Formula

$$Entropy(S) = -p_+ \log_2(p_+) - p_- \log_2(p_-)$$

$$Entropy(S) = -\frac{9}{14} \log_2\left(\frac{9}{14}\right) - \frac{5}{14} \log_2\left(\frac{5}{14}\right)$$

$$Entropy(S) = -(0.643)(-0.64) - (0.357)(-1.48)$$

$$Entropy(S) = 0.94$$

Total Entropy = 0.94

Step 2: Information Gain for “Outlook”

Split by Outlook

1) Sunny

- Yes = 2
- No = 3
- Total = 5

$$Entropy(Sunny) = -\frac{2}{5} \log_2 \frac{2}{5} - \frac{3}{5} \log_2 \frac{3}{5}$$

$$Entropy(Sunny) = 0.97$$

2) Overcast

- Yes = 4
- No = 0

$$Entropy(Overcast) = 0$$

3) Rain

- Yes = 3
- No = 2

$$Entropy(Rain) = 0.97$$

Weighted Entropy of Outlook

$$Entropy(Outlook) = \frac{5}{14}(0.97) + \frac{4}{14}(0) + \frac{5}{14}(0.97)$$

$$Entropy(Outlook) = 0.69$$

Information Gain

$$IG(Outlook) = Entropy(S) - Entropy(Outlook)$$

$$IG(Outlook) = 0.94 - 0.69 = 0.25$$

Step 3: (Quick Comparison) IG of Other Attributes

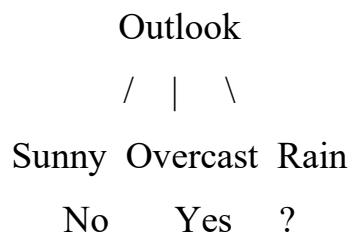
Attribute	Information Gain
Outlook	0.25
Temperature	0.03
Humidity	0.15
Wind	0.04

Step 4: Root Node Selection

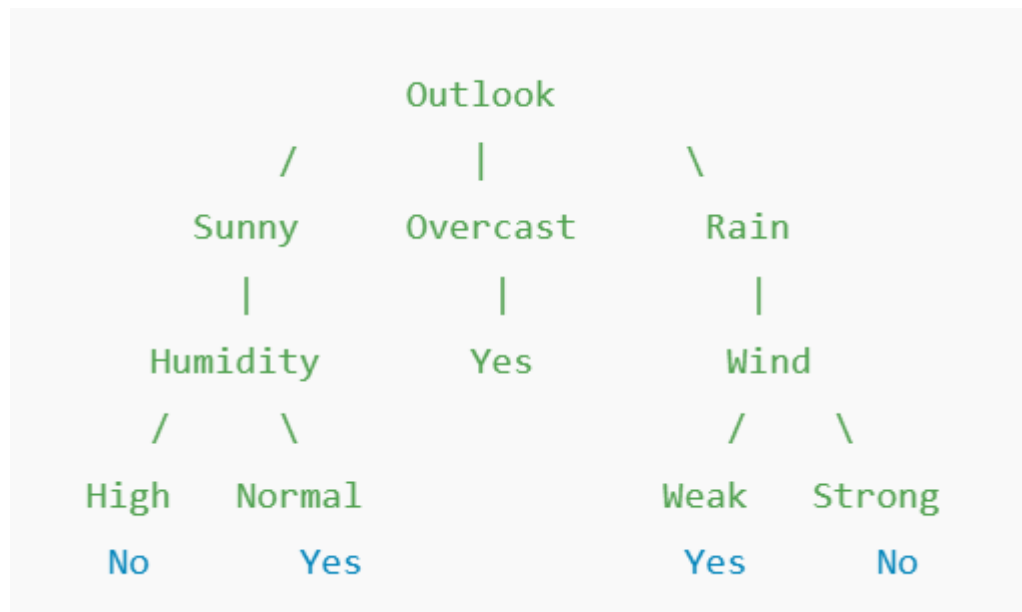
Outlook has the highest Information Gain

Root node = Outlook

Step 5: Final Decision Tree (Partial)



Further splitting is done for **Sunny** and **Rain** using the same steps will give



If Outlook = Overcast → Play Tennis = YES

If Outlook = Sunny AND Humidity = High → NO

If Outlook = Sunny AND Humidity = Normal → YES

If Outlook = Rain AND Wind = Weak → YES

If Outlook = Rain AND Wind = Strong → NO

Key Notes

- Entropy = measure of impurity
- Information Gain = reduction in entropy
- Highest IG → chosen as split

- Pure node entropy = 0

b) DECISION TREE USING GINI INDEX

Gini Index Formula

$$Gini(S) = 1 - \sum p_i^2$$

Lower Gini → better split

Minimum Gini → chosen attribute

Step 1: Gini of the Root Dataset

Play Tennis counts

- Yes = 9, No = 5
- Total = 14

$$\begin{aligned} Gini(S) &= 1 - \left(\frac{9}{14}\right)^2 - \left(\frac{5}{14}\right)^2 \\ &= 1 - (0.41 + 0.13) = 0.46 \end{aligned}$$

Gini(S) = 0.46

Step 2: Gini for Attribute Outlook

Split by Outlook

Sunny (5)

- Yes = 2, No = 3

$$Gini(Sunny) = 1 - (2/5)^2 - (3/5)^2 = 1 - (0.16 + 0.36) = 0.48$$

Overcast (4)

- Yes = 4, No = 0

$$Gini(Overcast) = 0$$

Rain (5)

- Yes = 3, No = 2

$$Gini(Rain) = 1 - (3/5)^2 - (2/5)^2 = 1 - (0.36 + 0.16) = 0.48$$

Weighted Gini for Outlook

$$\begin{aligned} Gini(Outlook) &= \frac{5}{14} (0.48) + \frac{4}{14} (0) + \frac{5}{14} (0.48) \\ &= 0.34 \end{aligned}$$

Step 3: Gini for Other Attributes (Summary)

Attribute	Weighted Gini
Outlook	0.34 (min)
Temperature	0.44
Humidity	0.37
Wind	0.43

Root Node = Outlook

Branch-wise Splitting

Branch 1: Outlook = Overcast

- Gini = 0
- Pure node

Leaf → YES

Branch 2: Outlook = Sunny

Yes = 2, No = 3

$$Gini(Sunny) = 0.48$$

Check Humidity

High

- Yes = 0, No = 3
- Gini = 0

Normal

- Yes = 2, No = 0
- Gini = 0

Weighted Gini

$$Gini(Humidity | Sunny) = \frac{3}{5}(0) + \frac{2}{5}(0) = 0$$

Best split = Humidity

Result

Sunny

└── Humidity

 └── High → No

 └── Normal → Yes

Branch 3: Outlook = Rain

Yes = 3, No = 2

$$Gini(Rain) = 0.48$$

Check Wind

Weak

- Yes = 3, No = 0
- Gini = 0

Strong

- Yes = 0, No = 2
- Gini = 0

Weighted Gini

$$Gini(Wind | Rain) = \frac{3}{5}(0) + \frac{2}{5}(0) = 0$$

Best split = Wind

Result

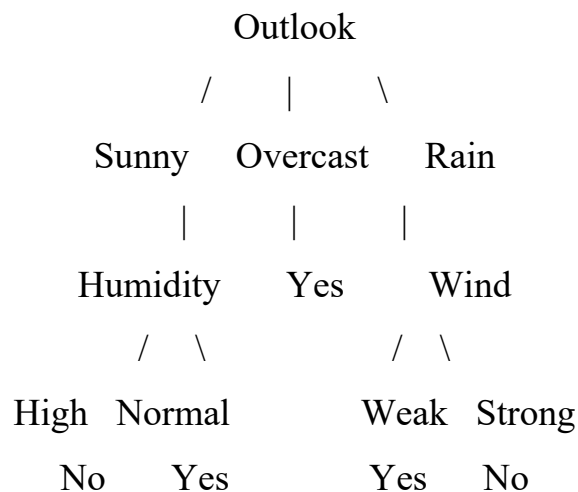
Rain

└─ Wind

└─ Weak → Yes

└─ Strong → No

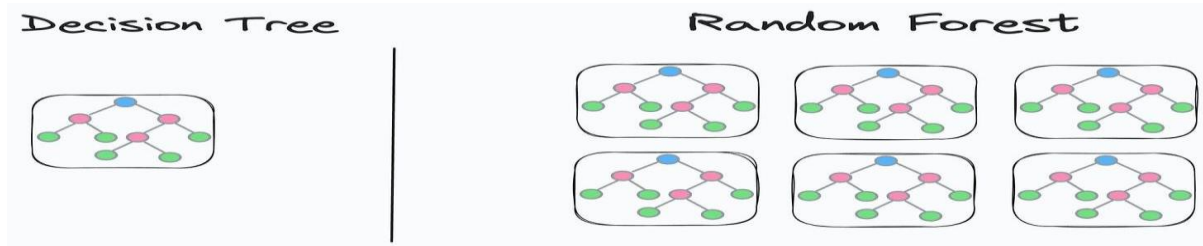
FINAL DECISION TREE (GINI)



- Entropy → maximize Information Gain
- Gini → minimize Gini Index
- Both often give same tree
- CART uses Gini
- ID3 uses Entropy

2) Random Forest Algorithm in Machine Learning

Random Forest is an ensemble learning algorithm that builds multiple decision trees and combines their outputs to improve accuracy and reduce overfitting.

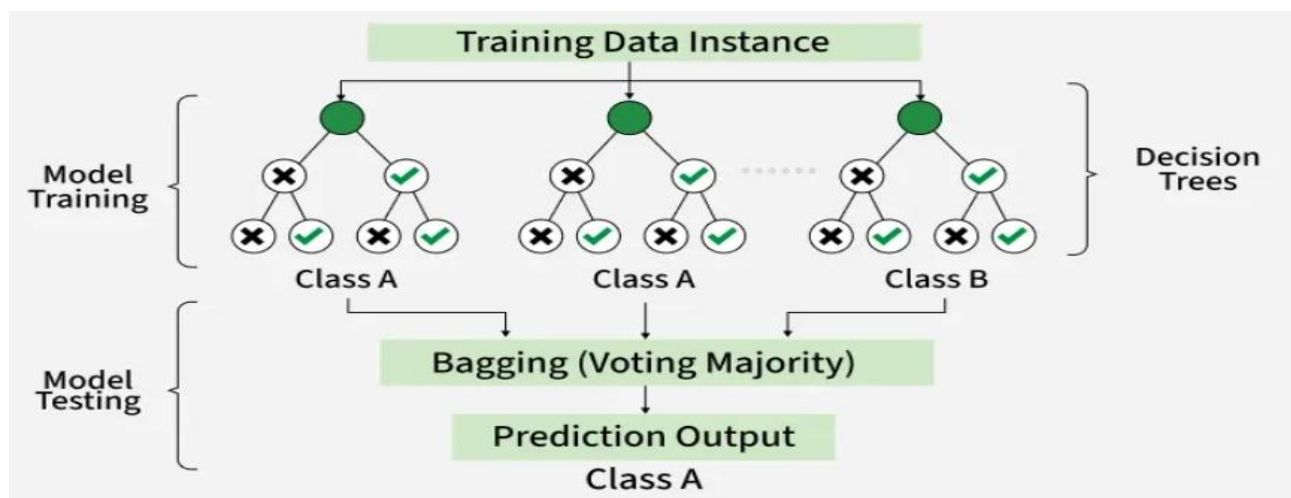


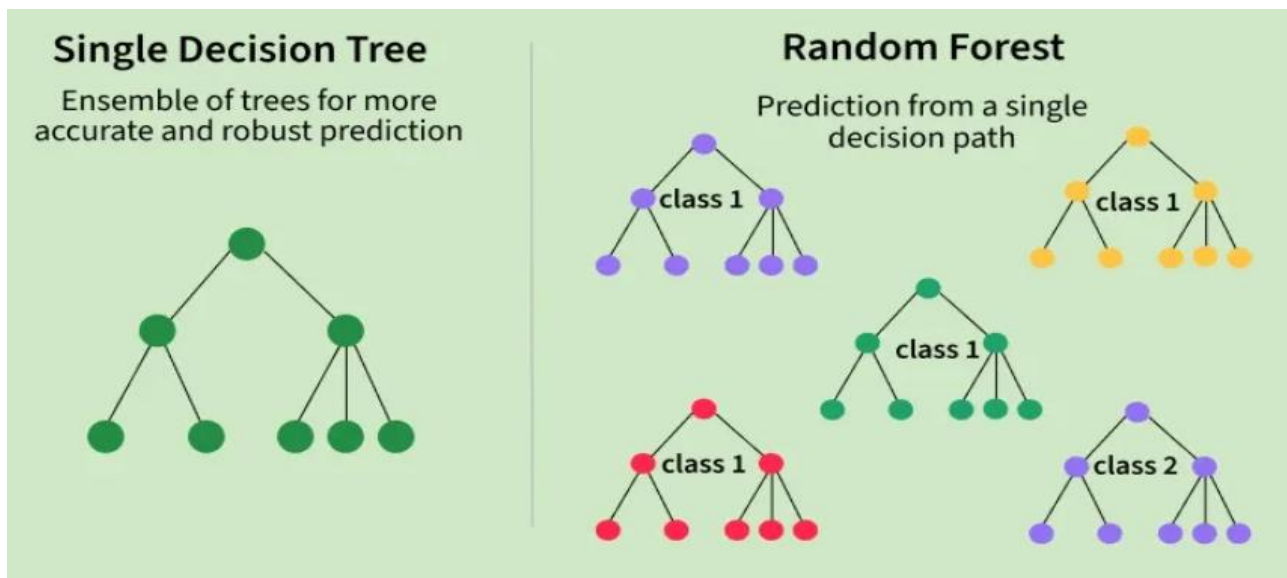
2.1 How it Works

1. Random sampling (Bootstrap): Multiple datasets are created by sampling the training data with replacement.
2. Random feature selection: At each split, only a random subset of features is considered.
3. Tree building: Each dataset trains a decision tree independently.
4. Prediction:
 - Classification: Final output by majority voting
 - Regression: Final output by average of predictions

2.2 Why “Random”?

- Random data sampling
 - Random feature selection at each split
- This **decorrelates trees** and improves performance.





2.3 Bootstrapping

Bootstrapping is a sampling technique where multiple datasets are created from the original dataset by sampling with replacement.

◇ Key Points

- Same data point can appear multiple times
- Some points may not appear at all
- Each sample is the same size as the original dataset
- Used to create diverse training sets

Bagging (Bootstrap Aggregating)

Bagging is an ensemble technique that:

- Trains multiple models on bootstrapped datasets
- Combines their predictions to produce a final output

Working

1. Create multiple datasets using **bootstrapping**
2. Train a model on each dataset (e.g., decision tree)
3. Combine results:
 - **Classification:** majority voting
 - **Regression:** averaging

Relationship Between Them

Term	Meaning
Bootstrapping	Data sampling technique
Bagging	Model training + aggregation method

Bootstrapping is used inside Bagging

Advantages

- Reduces **overfitting**
- Improves **stability and accuracy**
- Works well with **high-variance models** (like decision trees)

2.4 Advantages of Random Forest

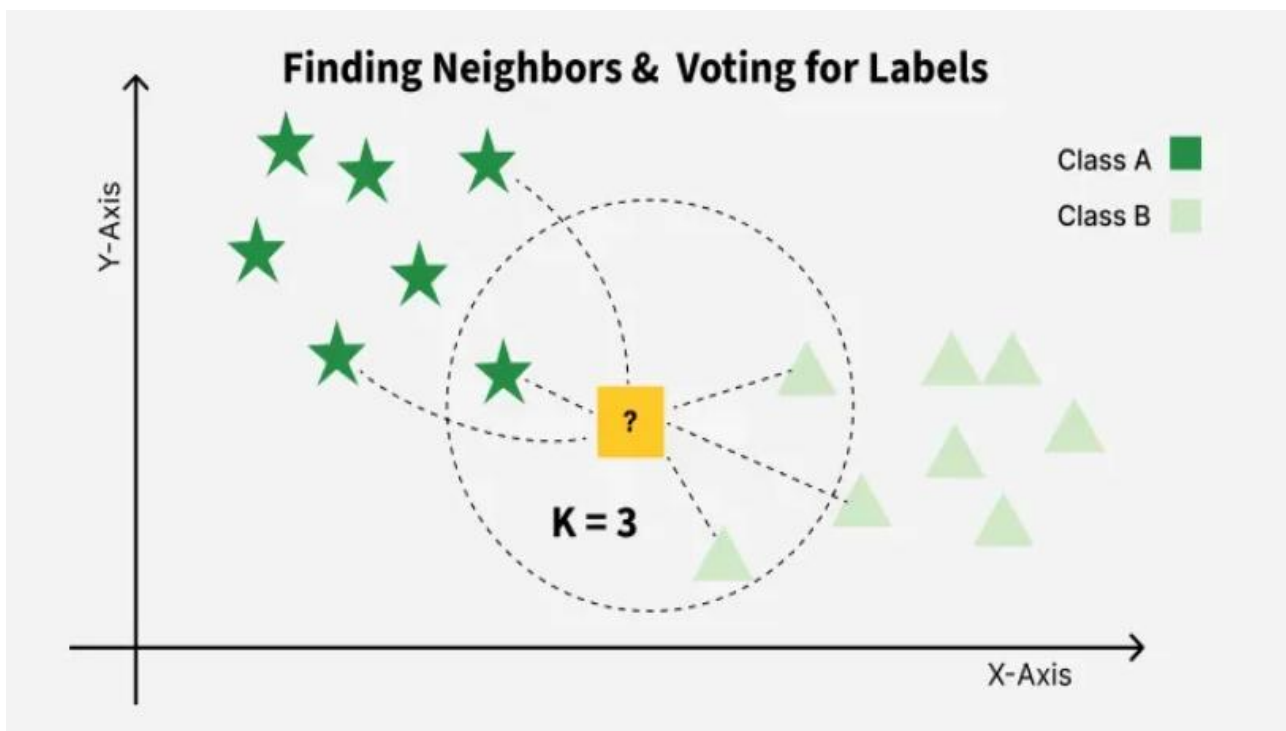
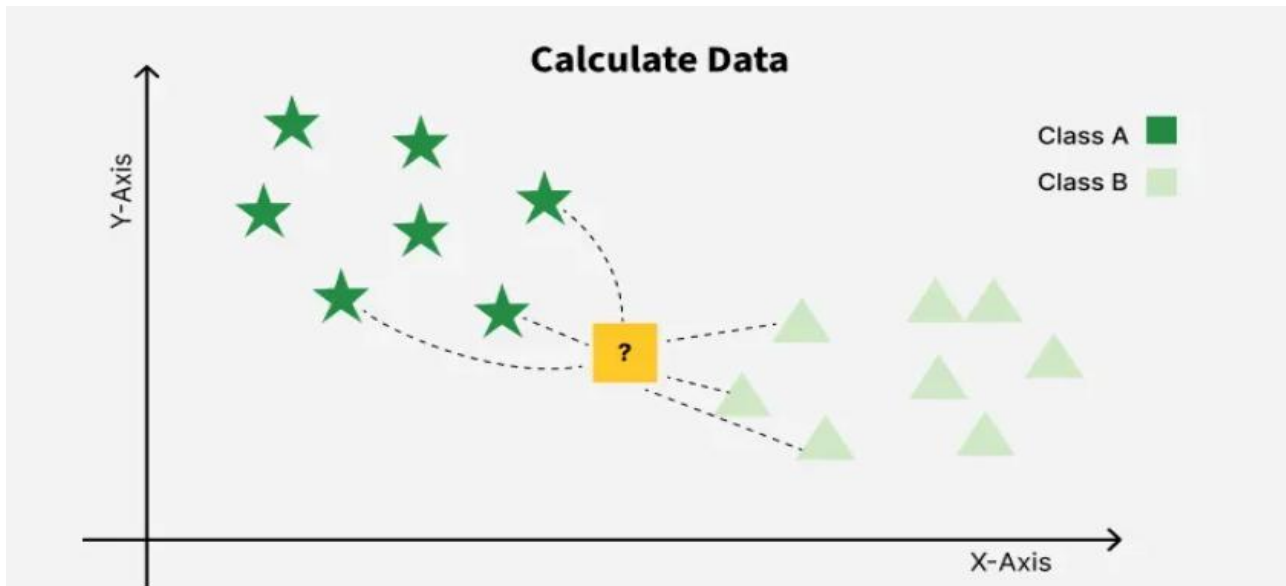
- Random Forest provides very accurate predictions even with large datasets.
- Random Forest can handle missing data well without compromising with accuracy.
- It doesn't require normalization or standardization on dataset.
- When we combine multiple decision trees it reduces the risk of overfitting of the model.

2.5 Limitations of Random Forest

- It can be computationally expensive especially with a large number of trees.
- It's harder to interpret the model compared to simpler models like decision trees.

3. k-Nearest Neighbors (k-NN)

K-Nearest Neighbors (KNN) is a supervised machine learning algorithm generally used for classification but can also be used for regression tasks. It works by finding the "k" closest data points (neighbors) to a given input and makes a predictions based on the majority class (for classification) or the average value (for regression). Since KNN makes no assumptions about the underlying data distribution it makes it a non-parametric and instance-based learning method.



3.1 Working Principle

1. Choose value of k
2. Calculate distance (usually Euclidean)
3. Find k nearest points

4. Majority vote decides class

Distance Formula

$$d = \sqrt{(x_1 - x_2)^2 + (y_1 - y_2)^2}$$

3.2 What is 'K' in K Nearest Neighbour?

In the k-Nearest Neighbours algorithm k is just a number that tells the algorithm how many nearby points or neighbors to look at when it makes a decision.

Example: Imagine you're deciding which fruit it is based on its shape and size. You compare it to fruits you already know.

- If $k = 3$, the algorithm looks at the 3 closest fruits to the new one.
- If 2 of those 3 fruits are apples and 1 is a banana, the algorithm says the new fruit is an apple because most of its neighbors are apples.

3.3 How to choose the value of k for KNN Algorithm?

- The value of k in KNN decides how many neighbors the algorithm looks at when making a prediction.
- Choosing the right k is important for good results.
- If the data has lots of noise or outliers, using a larger k can make the predictions more stable.
- But if k is too large the model may become too simple and miss important patterns and this is called underfitting.
- So k should be picked carefully based on the data.

3.4 Distance Metrics Used in KNN Algorithm

KNN uses distance metrics to identify nearest neighbor, these neighbors are used for classification and regression task. To identify nearest neighbor we use below distance metrics:

a) Euclidean Distance

Euclidean distance is defined as the straight-line distance between two points in a plane or space. You can think of it like the shortest path you would walk if you were to go directly from one point to another.

$$\text{distance}(x, X_i) = \sqrt{\sum_{j=1}^d (x_j - X_{ij})^2}$$

b) Manhattan Distance

This is the total distance you would travel if you could only move along horizontal and vertical lines like a grid or city streets. It's also called "taxicab distance" because a taxi can only drive along the grid-like streets of a city.

$$d(x, y) = \sum_{i=1}^n |x_i - y_i|$$

c) Minkowski Distance

Minkowski distance is like a family of distances, which includes both Euclidean and Manhattan distances as special cases.

$$d(x, y) = \left(\sum_{i=1}^n (x_i - y_i)^p \right)^{\frac{1}{p}}$$

From the formula above, when $p=2$, it becomes the same as the Euclidean distance formula and when $p=1$, it turns into the Manhattan distance formula. Minkowski distance is essentially a flexible formula that can represent either Euclidean or Manhattan distance depending on the value of p .

3.5 Advantages

- Simple and intuitive
- No training phase

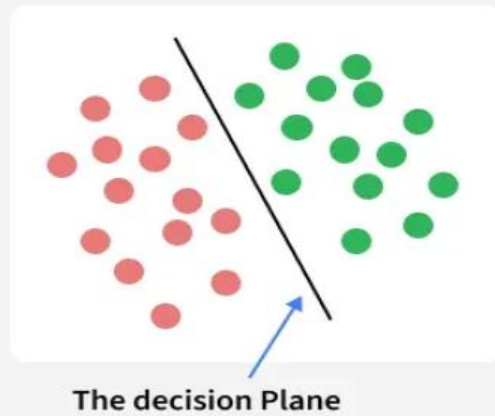
3.6 Disadvantages

- Slow for large datasets
- Sensitive to noise

4) Support Vector Machine (SVM) Algorithm

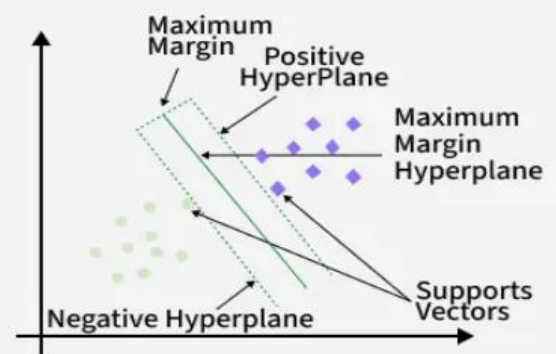
Support Vector Machine (SVM) is a supervised machine learning algorithm used for classification and regression tasks. It tries to find the best boundary known as hyperplane that separates different classes in the data. It is useful when you want to do binary classification like spam vs. not spam or cat vs. dog.

- Support Vector Machine is used for classification and regression.
- It works by finding decision planes that separate data into different classes.

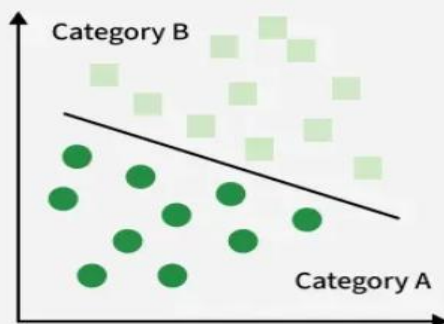


Support Vectors & Hyperplane

- Support Vectors are the closest data points to the hyperplane that define the class boundary.
- A hyperplane is a plane that separates different classes.

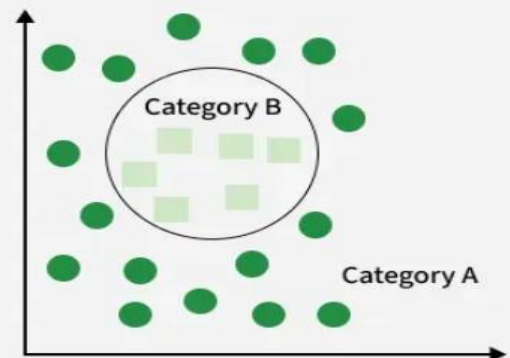


Linear SVM



Vs

Non-Linear SVM



The main goal of SVM is to maximize the margin between the two classes. The larger the margin the better the model performs on new and unseen data.

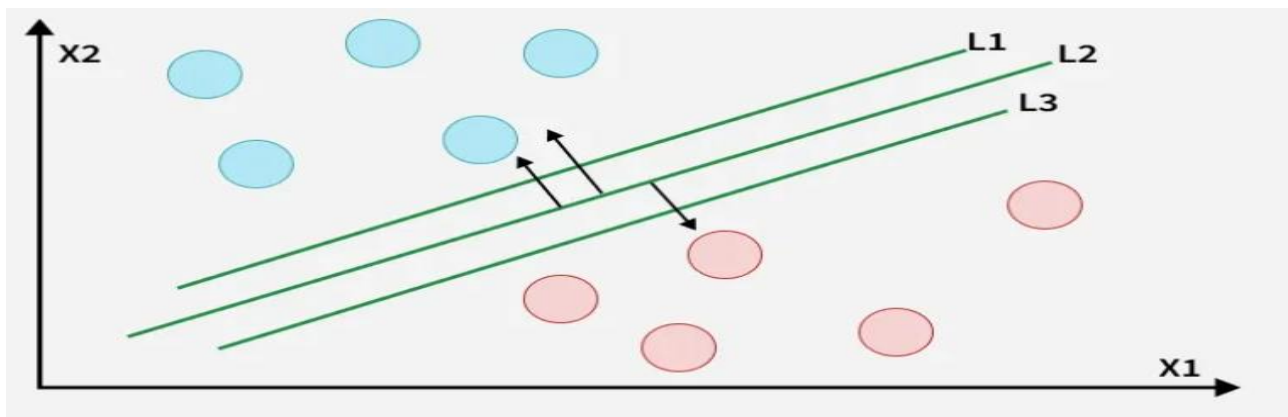
4.1 Key Concepts of Support Vector Machine

- **Hyperplane:** A decision boundary separating different classes in feature space and is represented by the equation $wx + b = 0$ in linear classification.
- **Support Vectors:** The closest data points to the hyperplane, crucial for determining the hyperplane and margin in SVM.

- **Margin:** The distance between the hyperplane and the support vectors. SVM aims to maximize this margin for better classification performance.
- **Kernel:** A function that maps data to a higher-dimensional space enabling SVM to handle non-linearly separable data.

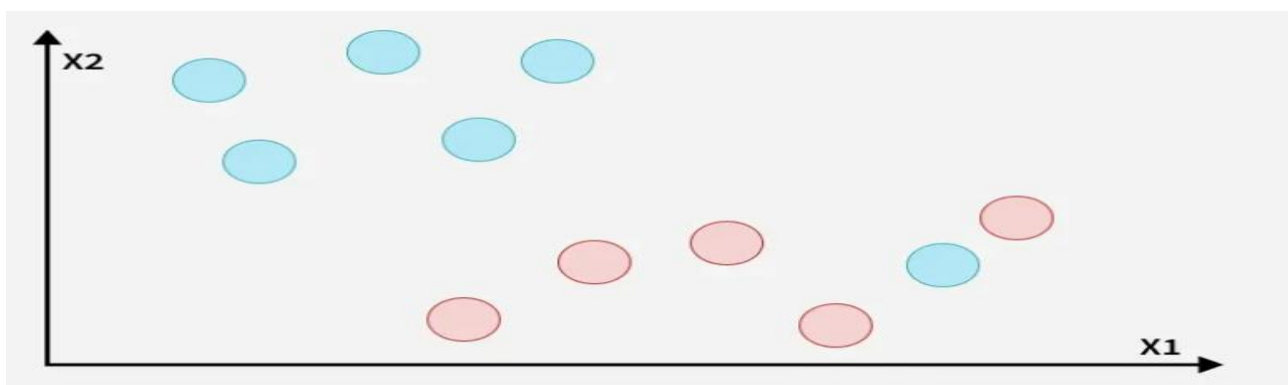
4.2 How does Support Vector Machine Algorithm Work?

The key idea behind the SVM algorithm is to find the hyperplane that best separates two classes by maximizing the margin between them. This margin is the distance from the hyperplane to the nearest data points (support vectors) on each side.



Multiple hyperplanes separate the data from two classes

The best hyperplane also known as the "**hard margin**" is the one that maximizes the distance between the hyperplane and the nearest data points from both classes. This ensures a clear separation between the classes. So from the above figure, we choose L2 as hard margin. Let's consider a scenario like shown below:

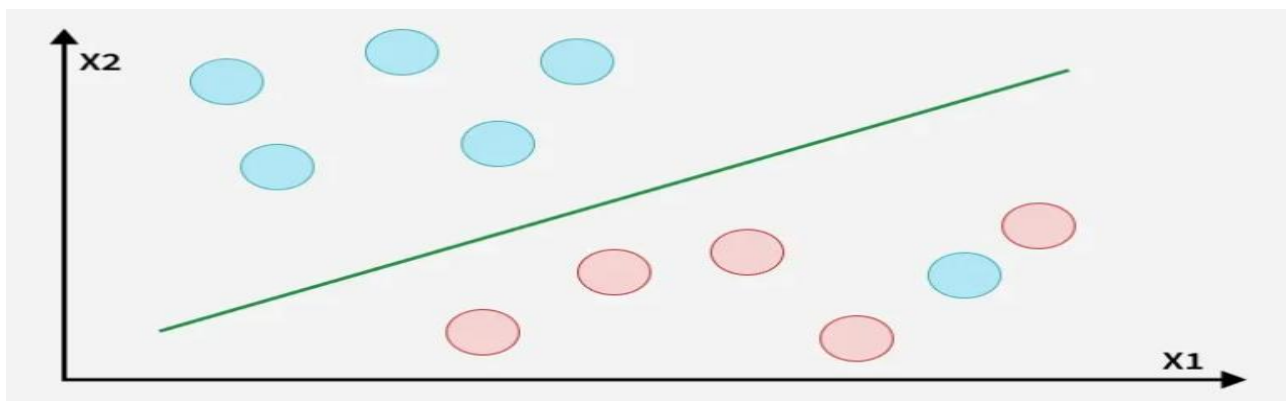


Selecting hyperplane for data with outlier

Here, we have one blue ball in the boundary of the red ball.

4.3 How does SVM classify the data?

The blue ball in the boundary of red ones is an outlier of blue balls. The SVM algorithm has the characteristics to ignore the outlier and finds the best hyperplane that maximizes the margin. SVM is robust to outliers.



Hyperplane which is the most optimized one

A soft margin allows for some misclassifications or violations of the margin to improve generalization. The SVM optimizes the following equation to balance margin maximization and penalty minimization:

$$\text{Objective Function} = \left(\frac{1}{\text{margin}}\right) + \lambda \sum \text{penalty}$$

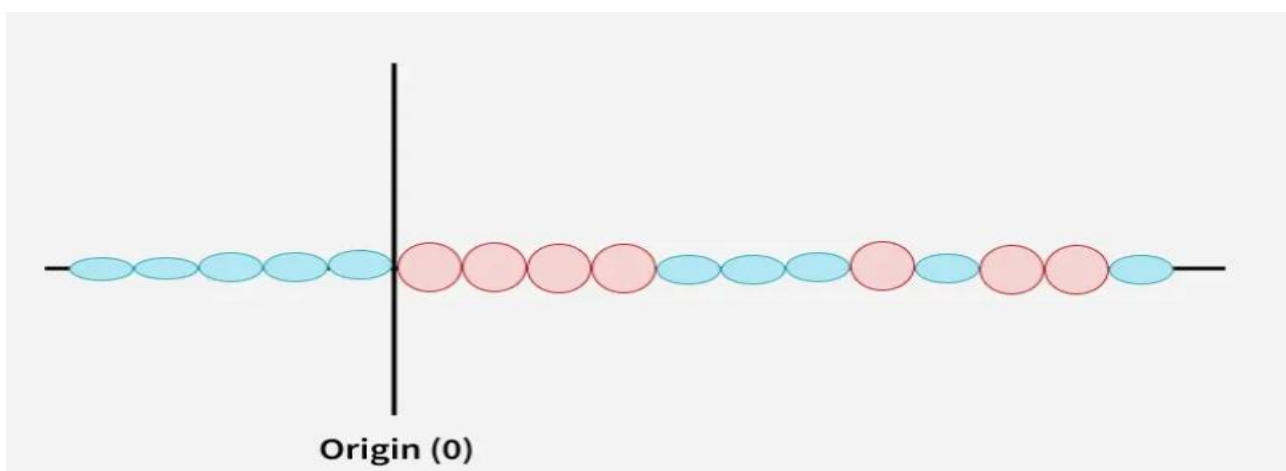
The penalty used for violations is often hinge loss which has the following behavior:

- If a data point is correctly classified and within the margin there is no penalty (loss = 0).
- If a point is incorrectly classified or violates the margin the hinge loss increases proportionally to the distance of the violation.

Till now we were talking about linearly separable data that separates group of blue balls and red balls by a straight line/linear line.

4.4 What if data is not linearly separable?

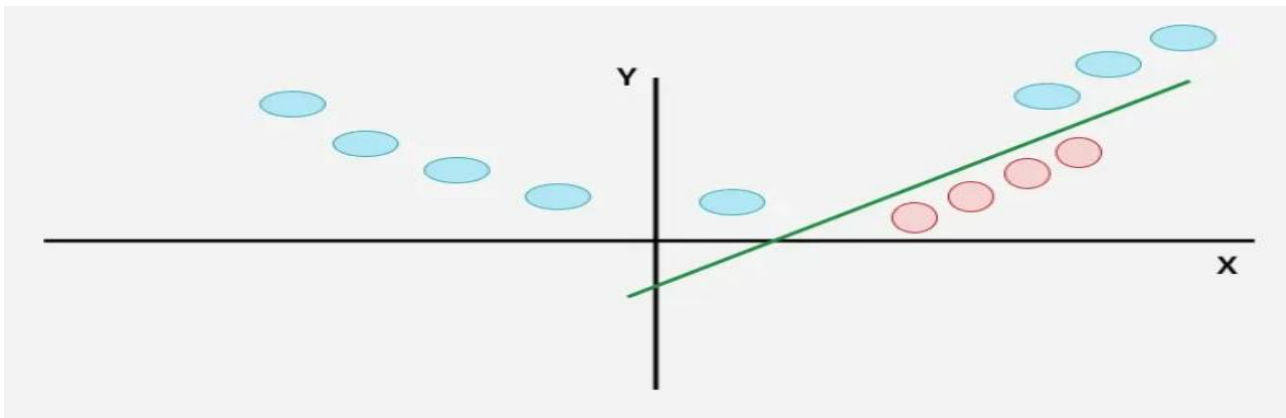
When data is not linearly separable i.e it can't be divided by a straight line, SVM uses a technique called kernels to map the data into a higher-dimensional space where it becomes separable.



This transformation helps SVM find a decision boundary even for non-linear data. Original 1D dataset for classification

A kernel is a function that maps data points into a higher-dimensional space without explicitly computing the coordinates in that space. This allows SVM to work efficiently with non-linear data by implicitly performing the mapping. For example consider data points that are not linearly separable. By applying a kernel function SVM transforms the data points into a higher-dimensional space where they become linearly separable.

- **Linear Kernel:** For linear separability.
- **Polynomial Kernel:** Maps data into a polynomial space.
- **Radial Basis Function (RBF) Kernel:** Transforms data into a space based on distances between data points.



Mapping 1D data to 2D to become able to separate the two classes

In this case the new variable y is created as a function of distance from the origin.

4.5 Advantages of Support Vector Machine (SVM)

1. **High-Dimensional Performance:** SVM excels in high-dimensional spaces, making it suitable for image classification and gene expression analysis.
2. **Nonlinear Capability:** Utilizing kernel functions like RBF and polynomial SVM effectively handles nonlinear relationships.
3. **Outlier Resilience:** The soft margin feature allows SVM to ignore outliers, enhancing robustness in spam detection and anomaly detection.
4. **Binary and Multiclass Support:** SVM is effective for both binary classification and multiclass classification suitable for applications in text classification.
5. **Memory Efficiency:** It focuses on support vectors making it memory efficient compared to other algorithms.

4.6 Disadvantages of Support Vector Machine (SVM)

1. **Slow Training:** SVM can be slow for large datasets, affecting performance in SVM in data mining tasks.
2. **Parameter Tuning Difficulty:** Selecting the right kernel and adjusting parameters like C requires careful tuning, impacting SVM algorithms.
3. **Noise Sensitivity:** SVM struggles with noisy datasets and overlapping classes, limiting effectiveness in real-world scenarios.
4. **Limited Interpretability:** The complexity of the hyperplane in higher dimensions makes SVM less interpretable than other models.
5. **Feature Scaling Sensitivity:** Proper feature scaling is essential, otherwise SVM models may perform poorly.

5) Naive Bayes Classifiers

Naïve Bayes is a probabilistic supervised classification algorithm based on Bayes' Theorem, with a *naïve assumption* that features are conditionally independent given the class.

Despite this “naïve” assumption, it works well especially for:

- Text classification
- Spam filtering
- Sentiment analysis
- Medical diagnosis
- Document categorization

Core Idea:

Instead of asking:

“Given these features, what class should I choose?”

Naïve Bayes asks:

“Given this class, how likely are these features?”

Then it flips the probability using **Bayes' Theorem**.

5.1 Bayes' Theorem (Foundation)

$$P(C | X) = \frac{P(X | C) P(C)}{P(X)}$$

Where:

- $P(C | X) \rightarrow$ **Posterior probability** (what we want)
- $P(X | C) \rightarrow$ **Likelihood**
- $P(C) \rightarrow$ **Prior probability**
- $P(X) \rightarrow$ **Evidence** (constant for all classes)

For classification:

$P(X)$ can be ignored

So we maximize:

$$P(C | X) \propto P(C) P(X | C)$$

Why is it called “Naïve”?

Because it assumes:

$$P(x_1, x_2, \dots, x_n | C) = \prod_{i=1}^n P(x_i | C)$$

Each feature contributes independently to the class prediction.

Example:

5.2 How Naïve Bayes Classifies (Step-by-Step)

Step 1: Compute Prior Probability

$$P(C) = \frac{\text{Number of samples in class C}}{\text{Total samples}}$$

Step 2: Compute Likelihood

For each feature:

$$P(x_i | C)$$

Step 3: Multiply probabilities

$$P(C) \prod P(x_i | C)$$

Step 4: Choose class with maximum posterior

5.3 Types of Naïve Bayes Classifiers

a) Gaussian Naïve Bayes

Used when features are **continuous**.

Assumes:

$$P(x | C) = \frac{1}{\sqrt{2\pi\sigma^2}} e^{-\frac{(x-\mu)^2}{2\sigma^2}}$$

Use cases:

- Medical measurements
- Sensor data
- Credit score, income

b) Multinomial Naïve Bayes

Used for **discrete counts**.

Common in NLP:

- Word frequencies
- Bag-of-Words
- TF-IDF

Example:

- Spam detection
- News classification

c) Bernoulli Naïve Bayes

Used for **binary features**.

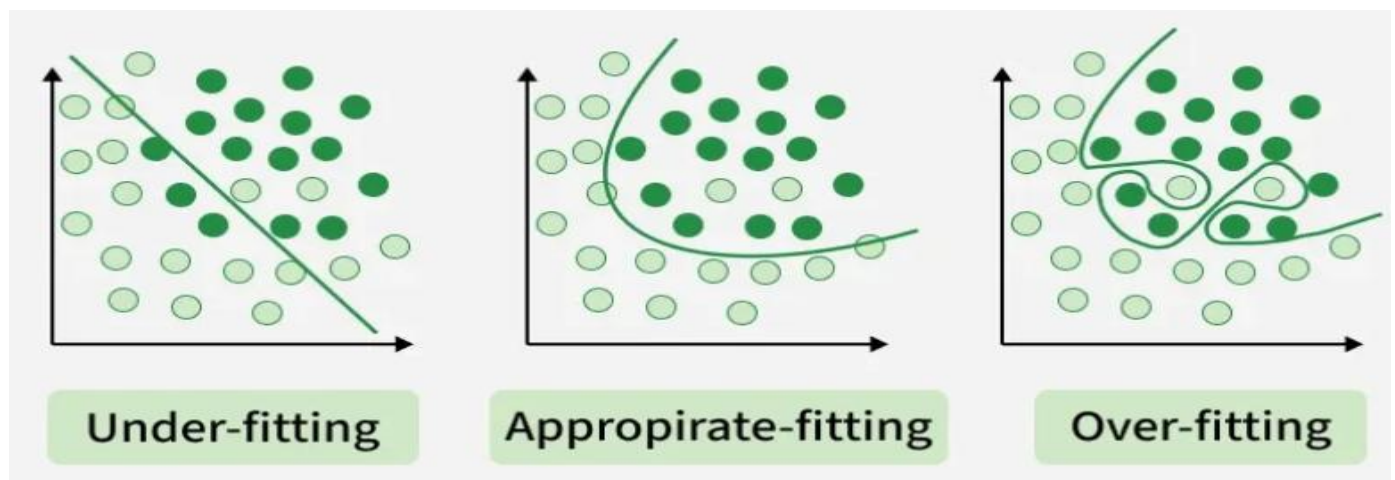
Example:

- Word present or not
- Clicked or not

7) Underfitting and Overfitting in ML

Machine learning models should learn useful patterns from training data. When a model learns too little or too much, we get underfitting or overfitting.

- Underfitting means that the model is too simple and does not cover all real patterns in the data.
- Overfitting means that the model learns not just the underlying pattern, but also noise or random quirks in the training data. model memorizes training data
- A good model finds the right *spot*, it is complex enough to capture real patterns, but not so complex that it “memorizes” noise



- **Underfitting (High Bias):** A model that is too simple (like a straight line for curved data) misses key patterns and performs poorly on both training and testing data.
- **Overfitting (High Variance):** A model that is too complex (like a high-degree polynomial) learns noise, fits training data too closely, and performs poorly on new data.
- **Ideal Fit (Balanced):** A moderately complex model captures the main trend without following noise, giving good performance on both training and testing data.

7.1 What is Underfitting?

Underfitting happens when the model fails to learn important patterns. It performs poorly on both training and testing data. Underfitting happens due to:

- Model is too simple
- Very high regularization
- Features are weak or missing
- Not enough training

Bias: *It is like assuming all birds can only be small and fly, so the model fails to recognize big birds like ostriches or penguins that can't fly and get biased with predictions.*

Bias Variance Inside Underfitting

Underfitting mainly occurs due to high bias:

- High bias means model makes strong assumptions
- Ignores patterns
- Learns an overly simple representation
- Variance is low because the model gives similar outputs even if the data changes

Underfitting = High Bias + Low Variance

7.2 What is Overfitting?

Overfitting happens when the model learns too much from the training data, including noise and outliers. It performs very well on training data but poorly on test data. Overfitting happens due to:

- Model too complex with many features
- Very little data
- No regularization
- High variance

***Variance:** Error that happens when a machine learning model learns too much from the data, including random noise.*

Bias Variance Inside Overfitting

Overfitting is mainly caused by high variance:

- High variance means model reacts too strongly to training data
- Learns noise as patterns
- Low bias because the model is extremely flexible

Overfitting = Low Bias + High Variance

7.3 Bias-Variance Tradeoff

The relationship between bias and variance is often referred to as the bias-variance tradeoff, which highlights the need for balance:

- Increasing model complexity reduces bias but increases variance (risk of overfitting).
- Simplifying the model reduces variance but increases bias (risk of underfitting).

The goal is to find an optimal balance where both bias and variance are minimized, resulting in good generalization performance.

7.4 How to Address Overfitting and Underfitting?

Techniques to Reduce Underfitting

- Use a more complex model
- Add new features and perform feature engineering
- Train for more epochs
- Scale features properly

Techniques to Reduce Overfitting

- Collect more training data
- Use regularization (L1/L2)
- Apply dropout (for neural networks)
- Use early stopping

7.5 Overfitting Control in Decision Trees

a) Why do Decision Trees Overfit?

Decision Trees keep splitting to reduce impurity (Gini / Entropy).

If unrestricted:

- They memorize training data
- Leaves may contain 1–2 samples
- Tree becomes deep and complex
- Training accuracy $\approx 100\%$
- Test accuracy drops

This is high variance.

b) Key Idea Behind Overfitting Control

Stop the tree from becoming too specific

This is done by:

- Limiting growth
- Removing weak splits
- Ensuring leaves represent enough data

c) Two Main Strategies

- **Pre-Pruning (Early Stopping)**
- **Post-Pruning (Tree Simplification)**

Pre-Pruning Techniques (Most Common in Practice)

These stop the tree **while it is being built**.

1. Maximum Depth (max_depth)

What it does:

Limits how deep the tree can grow.

Why it helps:

- Prevents too many splits
- Controls complexity

Scenario:

If depth is unlimited → memorization

If depth is small → general patterns

Example:

`DecisionTreeClassifier(max_depth=5)`

2. Minimum Samples per Split (min_samples_split)**What it does:**

A node must have at least **N samples** to split.

Why it helps:

- Prevents splitting on very small groups
- Reduces noise learning

Scenario:

If only 3 samples → splitting is unreliable

3. Minimum Samples per Leaf (min_samples_leaf)**What it does:**

Each leaf must contain at least **N samples**.

Why it helps:

- Smooths predictions
- Prevents tiny, noisy leaves

Very Important:

This is one of the **best controls for regression trees**.

4. Maximum Features (max_features)**What it does:**

Limits the number of features considered at each split.

Why it helps:

- Reduces dominance of strong features
- Adds randomness
- Lowers variance

Used heavily in Random Forests**5. Minimum Impurity Decrease****What it does:**

Split only if impurity reduction $\geq$ threshold.

Why it helps:

Avoids meaningless splits that don't add real information.

Post-Pruning (Cost Complexity Pruning)

Tree is:

1. Fully grown
2. Then pruned back

Cost-Complexity Pruning (α -pruning)**Idea:**

Balance:

- Tree accuracy
- Tree complexity

$$\text{Cost} = \text{Error} + \alpha \times \text{Tree Size}$$

- $\alpha \uparrow \rightarrow$ simpler tree
- $\alpha \downarrow \rightarrow$ complex tree

Effect:

Removes branches that contribute little to performance.

Q&A:

Q1) A k-NN model performs well on small datasets but becomes very slow on large ones. What causes this behavior?

Answer: k-NN is a lazy learner and stores all training data. During prediction, it computes distances to all points, increasing computation time.

Q2) Why does k-NN become unreliable when irrelevant features are added to the dataset?

Answer: Distance calculations get distorted, making nearest neighbors less meaningful and degrading prediction quality.

Q3) A decision tree keeps splitting until each leaf has very few samples. Why does this harm performance?

Answer: Excessive splitting causes overfitting by capturing noise instead of general patterns, leading to high variance.

Q4) Why are decision trees easy to explain but often unstable?

Answer: They follow clear if-else rules, making them interpretable, but small data changes can lead to completely different splits.

Q5) Replacing a single decision tree with a random forest improves accuracy. What fundamental principle causes this improvement?

Answer:

Random Forest reduces variance by averaging predictions from multiple trees trained on different subsets of data and features.

Q6) Why does Random Forest usually generalize better than a single decision tree?

Answer: Because errors made by individual trees cancel out when combined, reducing overfitting.

Q7) Why does SVM often perform well in high-dimensional feature spaces?

Answer: SVM focuses on maximizing the margin between classes and is less affected by the number of features.

Q8) simple model fails everywhere, while a very complex model works only on training data. What core concept explains both failures?

Answer: The bias–variance trade-off. High bias leads to underfitting, and high variance leads to overfitting. Good models balance both.

Q9) A decision tree shows very high training accuracy but poor test accuracy. What is this problem called and how can pruning help?

Answer: This is overfitting. Pruning removes unnecessary or weak branches from the tree, reducing complexity and helping the model generalize better to unseen data.

Q10) A decision tree performs poorly on both training and test data. What does this indicate and how can it be fixed?

Answer: This indicates underfitting. The model is too simple. It can be fixed by allowing a deeper tree, reducing pruning, or relaxing constraints like maximum depth.

Q11) Why does pruning a decision tree sometimes reduce training accuracy but improve test accuracy?

Answer: Pruning removes splits that capture noise in the training data. Although this lowers training accuracy, it reduces variance and improves performance on new data.

Q12) How does ensemble learning help overcome overfitting compared to a single decision tree?

Answer: Ensemble learning combines multiple models, so errors made by individual models cancel out. This reduces variance and improves generalization compared to a single overfitted tree.

Q13) Why do kernel functions allow SVMs to solve non-linear problems?

Answer: Kernels implicitly map data into higher-dimensional spaces where linear separation becomes possible without explicitly computing the transformation.

Q14) Why does Naïve Bayes work well with very small datasets?

Answer: It estimates only class-wise probabilities and requires fewer parameters, so it does not need large amounts of data to learn effectively.

Q12) Why does Naïve Bayes struggle when features are highly correlated?

Answer: Because correlated features violate the independence assumption, causing redundant information to be counted multiple times and reducing prediction reliability.